



Laboratorio di Programmazione

Lezione 8 – Puntatori II, Struct, Argomenti da linea di comando,
Redirezione `std[in/out]`

Marco Anisetti (teoria)

Dipartimento di Informatica

marco.anisetti@unimi.it

Matteo Luperto (lab. turno A)

Dipartimento di Informatica

matteo.luperto@unimi.it

Nicola Bena (lab. turno B)

Dipartimento di Informatica

nicola.bena@unimi.it

ESERCIZIO 1

Scambio di interi.

Tempo: 5 min.

Scrivete un programma che dichiari due puntatori di tipo `int` di nome `px` e `py`, assegni al contenuto delle locazioni di memoria puntate i valori 0 e 1, rispettivamente, e poi scambi i valori in modo che `px` punti al valore precedentemente puntato da `py` e viceversa. Per eseguire lo scambio usate un terzo puntatore a `int` ausiliario. Eseguite le stampe appropriate per verificare che lo scambio di valori sia avvenuto correttamente.

Soluzione Es. 1 Lezione 7

```
int x,y;
```

```
int *px = &x;
```

```
int *py = &y;
```

```
int *ptmp;
```

```
*px = 3;
```

```
*py = 5;
```

```
printf("Prima dello scambio:\n");
```

```
printf("px -> %d; py -> %d\n", *px, *py);
```

```
printf("x:%d y:%d\n",x,y);
```

```
ptmp = px;
```

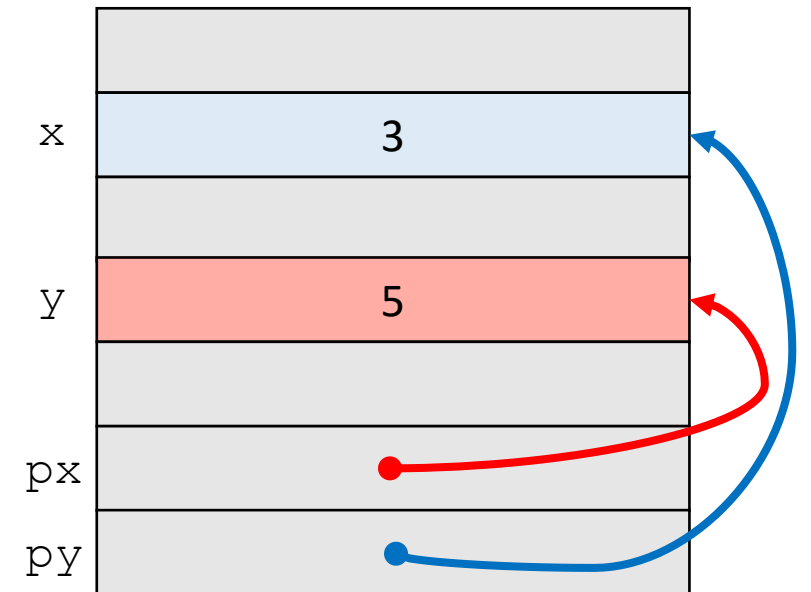
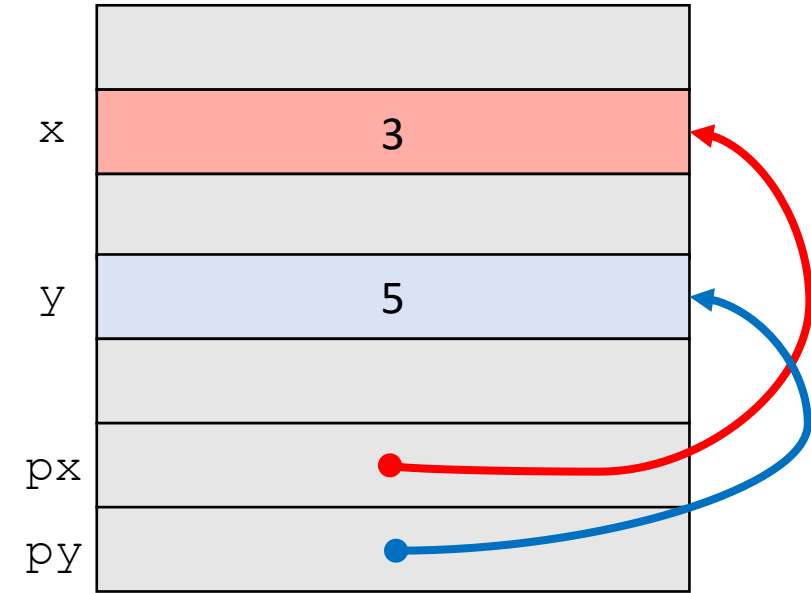
```
px = py;
```

```
py = ptmp;
```

```
printf("\nDopo lo scambio:\n");
```

```
printf("px -> %d; py -> %d\n", *px, *py);
```

```
printf("x:%d y:%d\n",x,y);
```



ESERCIZIO 1

Scambio di interi.

Tempo: 5 min.

Scrivete un programma che dichiari due puntatori di tipo `int` di nome `px` e `py`, assegni al contenuto delle locazioni di memoria puntate i valori 0 e 1, rispettivamente, e poi scambi i valori in modo che `px` punti al valore precedentemente puntato da `py` e viceversa. Per eseguire lo scambio usate un terzo puntatore a `int` ausiliario. Eseguite le stampe appropriate per verificare che lo scambio di valori sia avvenuto correttamente.

Vediamo ora una variante: scambiamo i valori di `x` e `y`, usando però i puntatori.

Scrivete un programma che dichiari due puntatori di tipo `int` di nome `px` e `py`, assegni al contenuto delle locazione di memoria puntate i valori 0 e 1, rispettivamente, e poi scambi i valori delle due locazioni di memoria senza andare a modificare i puntatori.

Di che formato deve essere la variabile ausiliaria che necessariamente serve per fare lo scambio?

Soluzione Es. 1 Lezione 7 variante

```
int x,y;
```

```
int *px = &x;
```

```
int *py = &y;
```

```
int tmp;
```

```
*px = 3;
```

```
*py = 5;
```

```
printf("Prima dello scambio:\n");
```

```
printf("px -> %d; py -> %d\n", *px, *py);
```

```
printf("x:%d y:%d\n",x,y);
```

```
tmp = *px;
```

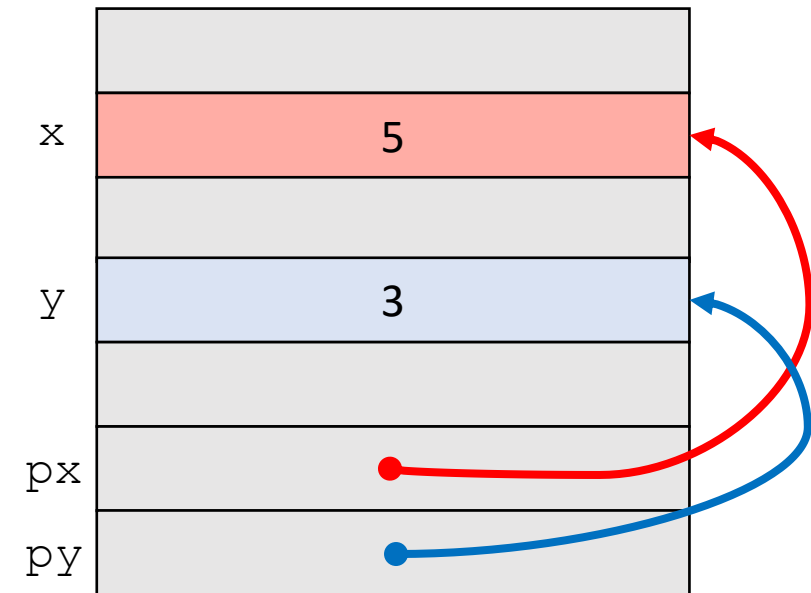
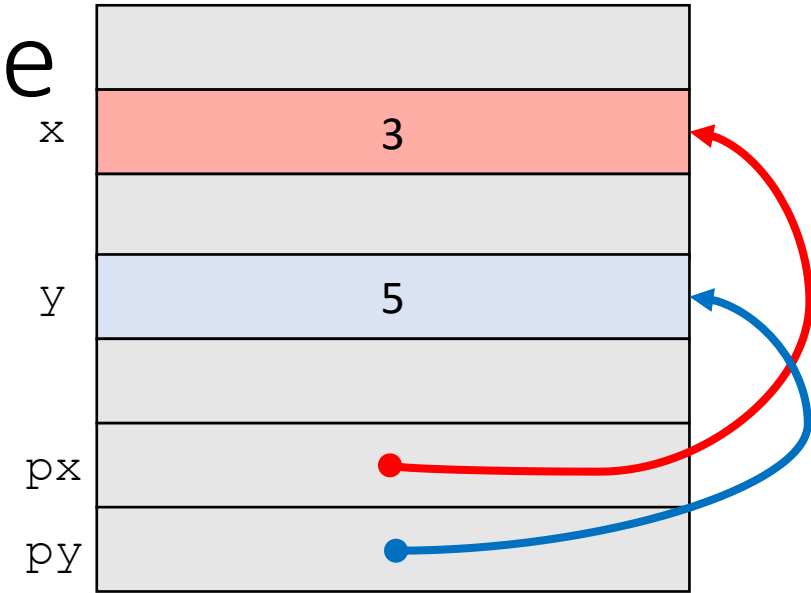
```
*px = *py;
```

```
*py = tmp;
```

```
printf("\nDopo lo scambio:\n");
```

```
printf("px -> %d; py -> %d\n", *px, *py);
```

```
printf("x:%d y:%d\n",x,y);
```



Aritmetica dei Puntatori

- L'indirizzo memorizzato da un puntatore è un numero
 - E' possibile usare operazioni aritmetiche e di confronto
- Dato un puntatore `p` e un intero `j`:
 - `p + j` vale `(int)p + sizeof(*p) * j`

Aritmetica dei Puntatori

- L'indirizzo memorizzato da un puntatore è un numero
 - E' possibile usare operazioni aritmetiche e di confronto
- Dato un puntatore `p` e un intero `j`:
 - `p + j` vale `(int)p + sizeof(*p) * j`
 - 1. Il valore del puntatore viene trattato come `(long) int`

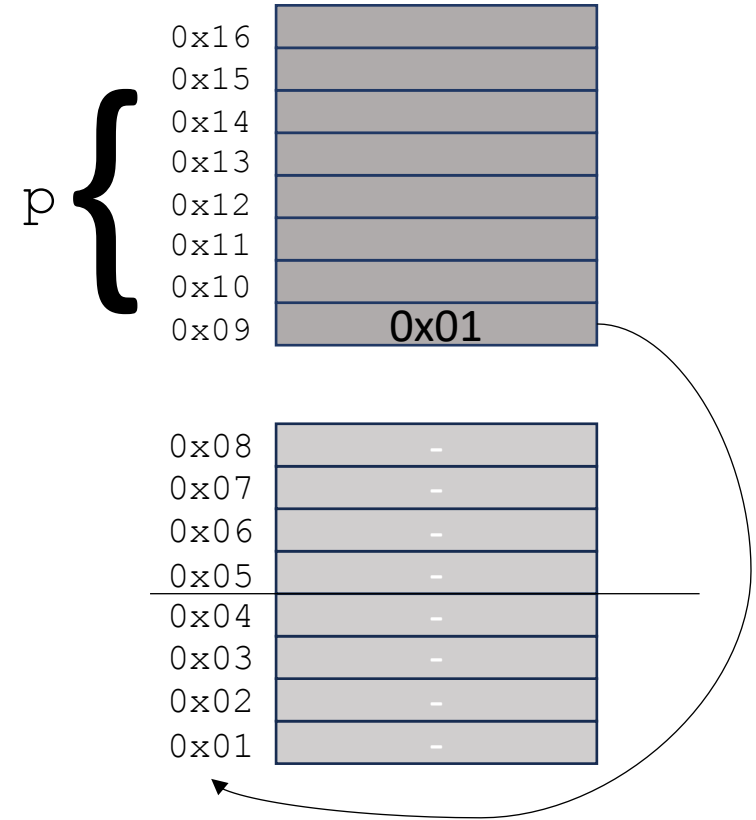
Aritmetica dei Puntatori

- L'indirizzo memorizzato da un puntatore è un numero
 - E' possibile usare operazioni aritmetiche e di confronto
- Dato un puntatore `p` e un intero `j`:
 - `p + j` vale `(int)p + sizeof(*p) * j`
 1. Il valore del puntatore `p` viene trattato come un intero
 2. Il valore di `j` corrisponde a `j` volte la dimensione dell'oggetto puntato da `p`

Aritmetica dei Puntatori

- Dato un puntatore p e un intero j :
 - $p + j$ vale $(\text{int})p + \text{sizeof}(*p) * j$
- Supponiamo $j=1$

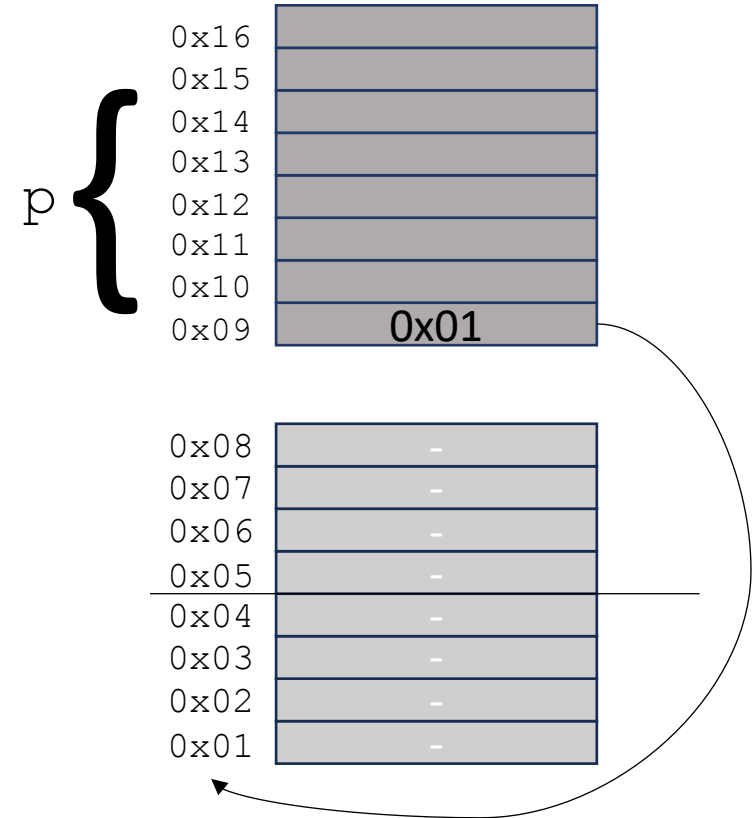
p punta a una variabile `int`
`sizeof(int) → 4`



Aritmetica dei Puntatori

- Dato un puntatore p e un intero j :
 - $p + j$ vale $(\text{int})p + \text{sizeof}(*p) * j$
- Supponiamo $j=1$
 1. Il contenuto di p viene trattato come `int`

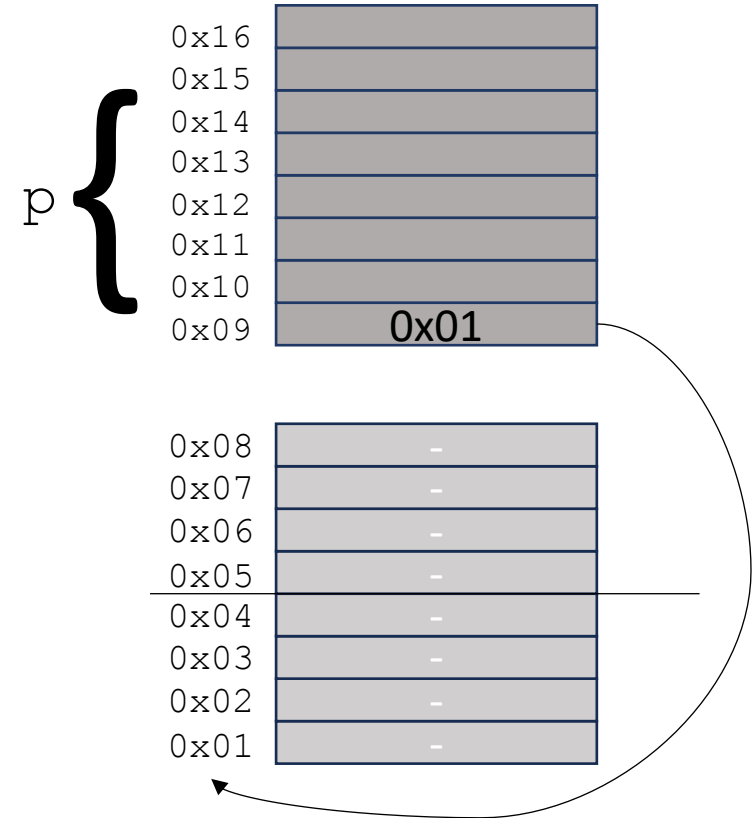
p punta a una variabile `int`
`sizeof(int) → 4`



Aritmetica dei Puntatori

- Dato un puntatore p e un intero j :
 - $p + j$ vale $(\text{int})p + \text{sizeof}(*p) * j$
- Supponiamo $j=1$
 1. Il contenuto di p viene trattato come $\text{int} \rightarrow 1$

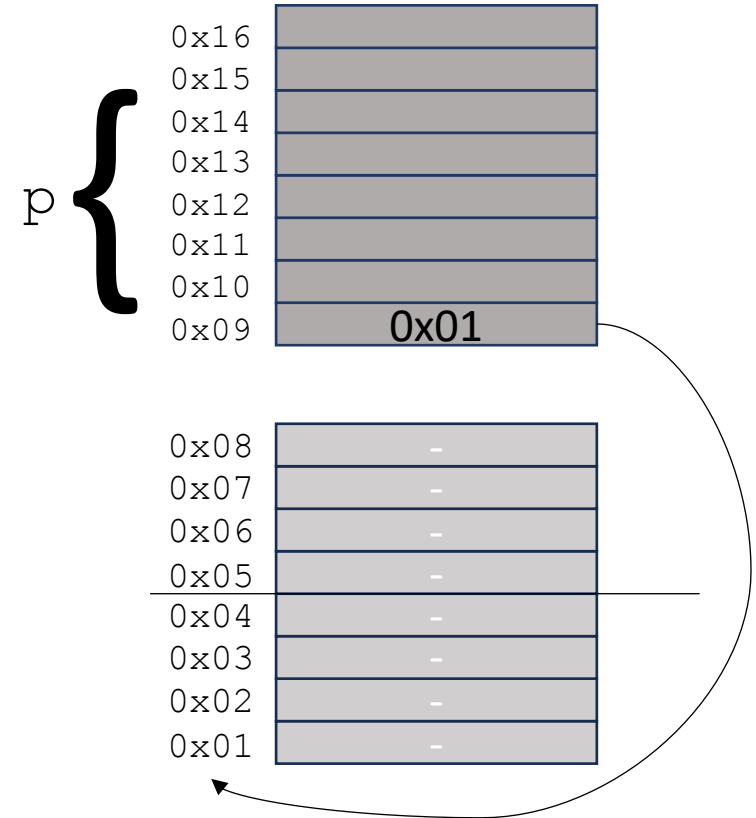
p punta a una variabile `int`
`sizeof(int)  $\rightarrow$  4`



Aritmetica dei Puntatori

- Dato un puntatore p e un intero j :
 - $p + j$ vale $(\text{int})p + \text{sizeof}(*p) * j$
- Supponiamo $j=1$
 1. Il contenuto di p viene trattato come $\text{int} \rightarrow 1$
 2. Il valore di j è $\text{sizeof}(*p) * j$

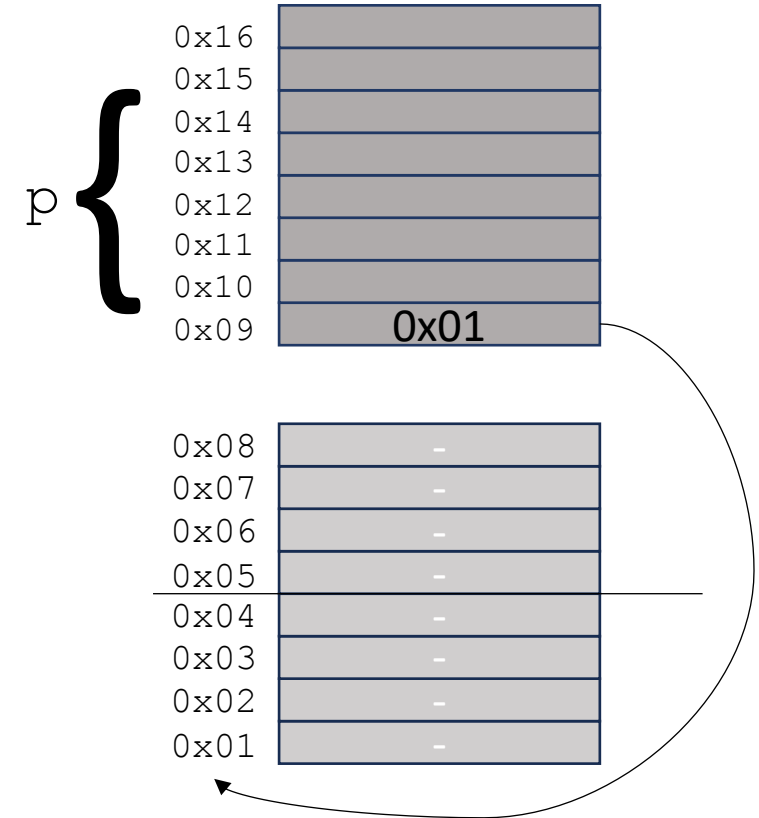
p punta a una variabile int
 $\text{sizeof}(\text{int}) \rightarrow 4$



Aritmetica dei Puntatori

- Dato un puntatore p e un intero j :
 - $p + j$ vale $(\text{int})p + \text{sizeof}(*p) * j$
- Supponiamo $j=1$
 1. Il contenuto di p viene trattato come $\text{int} \rightarrow 1$
 2. Il valore di j è $\text{sizeof}(*p) * j \rightarrow 4 * 1$

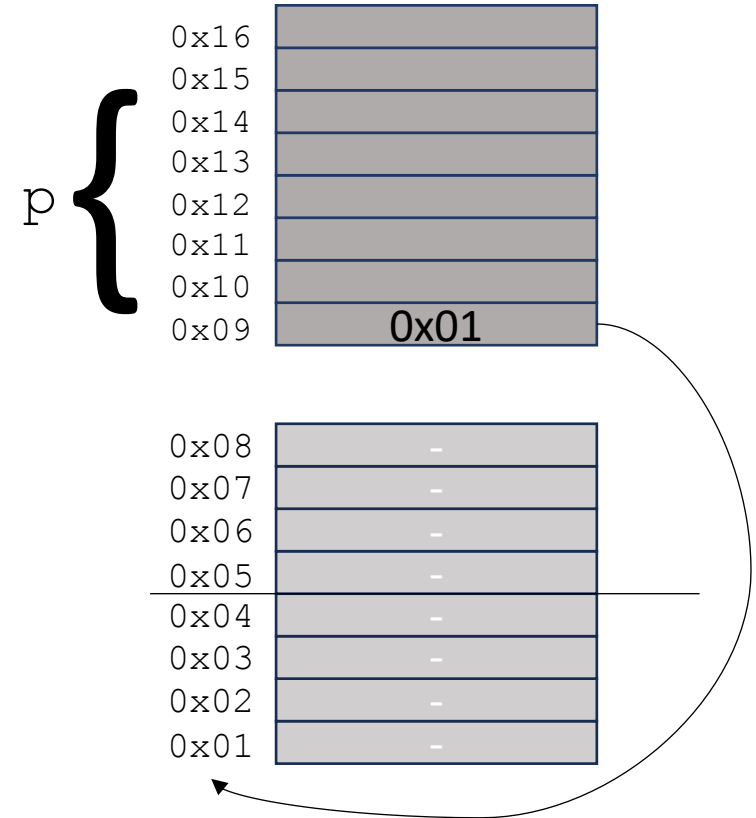
p punta a una variabile `int`
`sizeof(int)  $\rightarrow$  4`



Aritmetica dei Puntatori

- Dato un puntatore p e un intero j :
 - $p + j$ vale $(\text{int})p + \text{sizeof}(*p) * j$
- Supponiamo $j=1$
 1. Il contenuto di p viene trattato come $\text{int} \rightarrow 1$
 2. Il valore di j è $\text{sizeof}(*p) * j \rightarrow 4 * 1$
 3. Il valore di $(\text{int})p + j$

p punta a una variabile int
 $\text{sizeof}(\text{int}) \rightarrow 4$

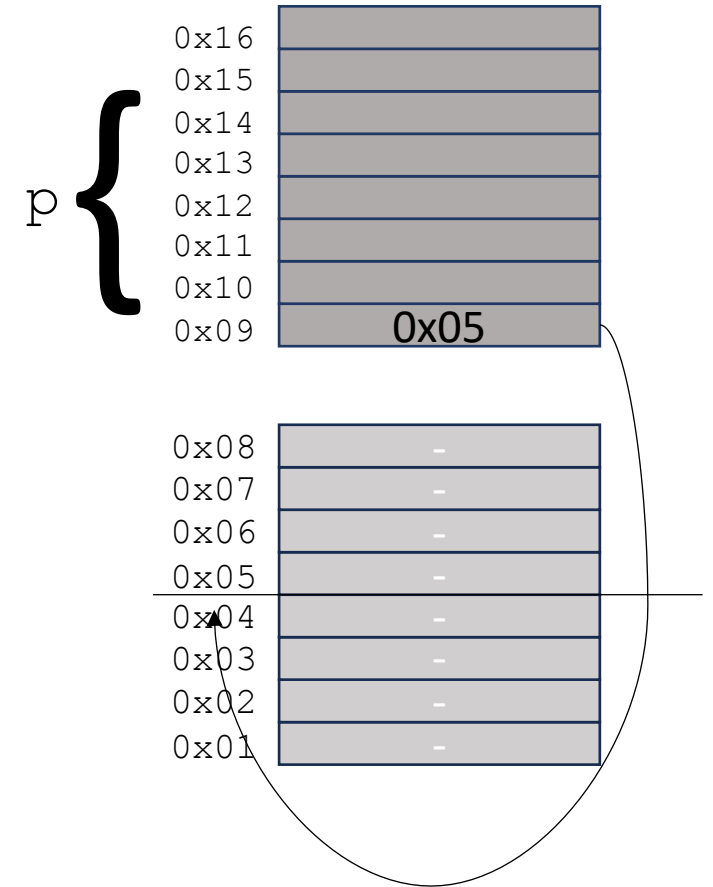


Aritmetica dei Puntatori

- Dato un puntatore p e un intero j :
 - $p + j$ vale $(\text{int})p + \text{sizeof}(*p) * j$
- Supponiamo $j=1$
 1. Il contenuto di p viene trattato come $\text{int} \rightarrow 1$
 2. Il valore di j è $\text{sizeof}(*p) * j \rightarrow 4 * 1$
 3. Il valore di $(\text{int})p + j \rightarrow 0x05$

→ Aggiungere 1 a un puntatore corrisponde a “spostarsi in avanti” del numero di celle necessarie a memorizzare una variabile del tipo puntato

p punta a una variabile int
 $\text{sizeof}(\text{int}) \rightarrow 4$

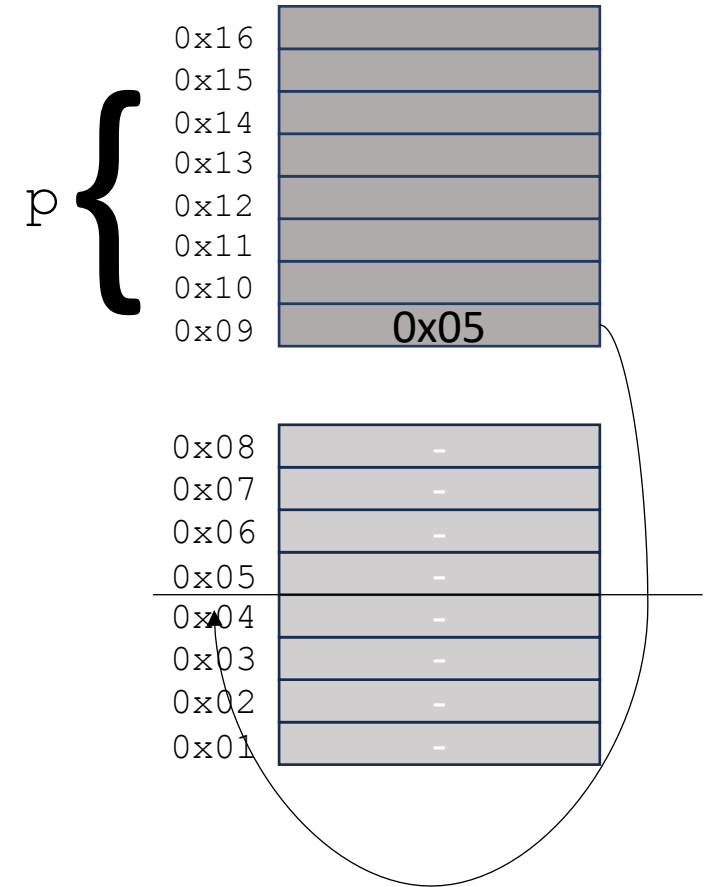


Aritmetica dei Puntatori

- Dato un puntatore p e un intero j :
 - $p + j$ vale $(\text{int})p + \text{sizeof}(*p) * j$
- Supponiamo $j=1$
 1. Il contenuto di p viene trattato come $\text{int} \rightarrow 1$
 2. Il valore di j è $\text{sizeof}(*p) * j \rightarrow 4 * 1$
 3. Il valore di $(\text{int})p + j \rightarrow 0x05$

→ Aggiungere X a un puntatore corrisponde a “spostarsi in avanti” del numero di celle necessarie a memorizzare X variabili del tipo puntato

p punta a una variabile int
 $\text{sizeof}(\text{int}) \rightarrow 4$



Aritmetica dei Puntatori

- L'indirizzo memorizzato da un puntatore è un numero
 - E' possibile usare operazioni aritmetiche e di confronto
- Dati due puntatori `p` e `q`:
 - `p - q = ((int) p - (int) q) / sizeof(*p)`

Aritmetica dei Puntatori

- L'indirizzo memorizzato da un puntatore è un numero
 - E' possibile usare operazioni aritmetiche e di confronto
- Dati due puntatori `p` e `q`:
 - `p - q = ((int) p - (int) q) / sizeof(*p)`
- Supponiamo `p=0x05` e `q=0x01`

Aritmetica dei Puntatori

- L'indirizzo memorizzato da un puntatore è un numero
 - E' possibile usare operazioni aritmetiche e di confronto
 - Dati due puntatori `p` e `q`:
 - $p - q = ((\text{int})\ p - (\text{int})\ q) / \text{sizeof}(*p)$
 - Supponiamo `p=0x05` e `q=0x01`, con `p` e `q` di tipo `int*`
 1. $((\text{int})\ p - (\text{int})\ q) \rightarrow 4$
 2. $4 / \text{sizeof}(\text{int}) \rightarrow 1$
- Nelle operazioni sui puntatori l'unità di base è `sizeof(<tipo oggetto puntato>)`

Esempio

```
#include <stdio.h>
#define N 25
int main()
{
    char str[N] = "Hello World";
    char *ptr = &str[0];
    printf("%li %li %li\n", str, ptr, ptr+1);
    printf("offset: %lu\n", sizeof(*ptr));
    printf("%c %c\n", *ptr, *(ptr+1));

    int arr[N] = {1,2,3,4,5,6,7,8};
    int *ptr2 = &arr[0];
    printf("%li %li %li\n", arr, ptr2, ptr2+1);
    printf("offset: %lu\n", sizeof(*ptr2));
    printf("%d %d\n", *ptr2, *(ptr2+1));
}
```

```
goldleaf@GoldBookPro2 L8 % ./a.out
6136459744 6136459744 6136459745
offset: 1
H e
6136459644 6136459644 6136459648
offset: 4
1 2
```

Un `char` è grande 1 byte, quindi aggiungere +1 ad un puntatore di `char` fa spostare il pointer di 1 byte.

Un `int` è grande 4 byte, quindi aggiungere +1 ad un puntatore di `int` fa spostare il pointer di 4 byte.

Aritmetica dei Puntatori

- L'aritmetica dei puntatori fornisce un modo alternativo per scorrere gli array
- Dato un puntatore p e un intero j
 - se $p = \&\text{array}[i]$, allora $p+j$ corrisponde a $\&\text{array}[i+j]$
- Dati due puntatori p e q :
 - se $p = \&\text{array}[i]$ e $q = \&\text{array}[j]$, allora $p-q$ corrisponde a $i-j$

Equivalenza Array – Puntatori

- In C una variable `a` di tipo array equivale a un puntatore al primo elemento dell'array per via dell'aritmetica dei puntatori
 - `a` equivale a `&a[0]`
 - `a+1` equivale a `&a[+1]`
 - ...

Equivalenza Array – Puntatori

- In C una variable `a` di tipo array equivale a un puntatore al primo elemento dell'array per via dell'aritmetica dei puntatori
 - `a` equivale a `&a[0]`
 - `a+1` equivale a `&a[+1]`
 - ...

```
int a[N];
```

```
int *p;
```

```
p = a; ←
```

Lecito in C, `p` punta allo stesso indirizzo cui punta `a`

Equivalenza Array – Puntatori

- In C una variable `a` di tipo array equivale a un puntatore al primo elemento dell'array per via dell'aritmetica dei puntatori
 - `a` equivale a `&a[0]`
 - `a+1` equivale a `&a[+1]`
 - ...

```
int a[N];
```

```
int *p;
```

```
p = a;
```

```
p[5] = 3 * 2;
```



Lecito in C: possiamo usare un puntatore come un vettore

Equivalenza Array – Puntatori

In C una variable `a` di tipo array equivale a un puntatore al primo elemento dell'array per via dell'aritmetica dei puntatori

- `a` equivale a `&a[0]`
- `a+1` equivale a `&a[+1]`
- ...

```
int a[N];
```

```
int *p;
```

```
p = a;
```

```
p[5] = 3 * 2;
```

Le parentesi `[]` sono *syntactic sugar*, ovvero costrutti sintattici che semplificano la vita del programmatore e sono tradotti in costrutti equivalenti ma più macchinosi/di basso livello

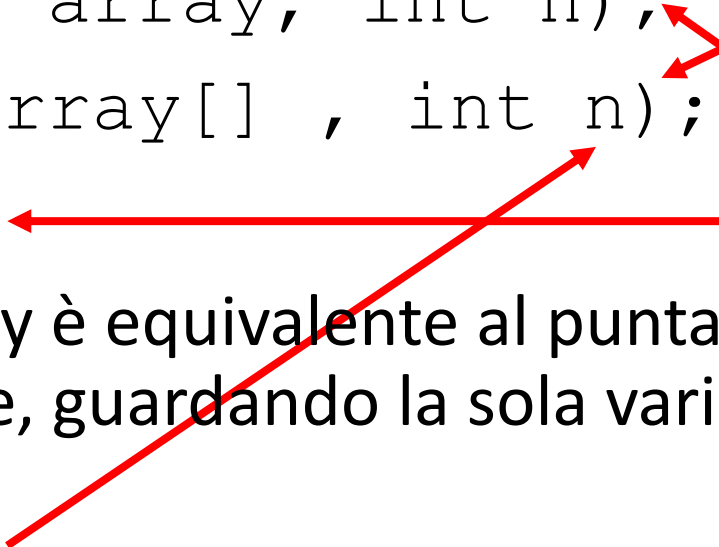
Funzioni e Array

Una funzione che prende in input un array prende in input il puntatore al primo elemento

```
void my_func(int * array, int n);  
void my_func(int array[], int n);  
my_func(array, 5);
```

Prototipi leciti

Chiamata



Attenzione: poiché un array è equivalente al puntatore alla sua prima cella, non possiamo sapere, guardando la sola variabile array, qual è la sua lunghezza

- Serve definire un parametro in cui specificare la lunghezza dell'array
- Un puntatore sa quanto grande ogni elemento dell'array, (`sizeof`) ma non quanti elementi contiene l'array

Matrici e Puntatori

Un array multidimensionale è un array di array, quindi il suo tipo è puntatore di array

- `int array[M][N]` equivale a `int (*) [N] array`
 - → sequenza contigua di celle
 - → la variabile punta a un array
- `int** array` indica una cosa simile (ma non uguale), cioè un array di puntatori a puntatori (prossime lezioni)
 - → sequenza non contigua di celle
- Funzioni:
 - `void my_func(int *array[])`
 - `void my_func(int array[][M])`

Serve specificare la grandezza di ogni riga (M) perché il compilatore deve sapere quanto è grande il dato puntato (la lunghezza di ogni riga)

Matrici e Puntatori

```
#include <stdio.h>
#define M 3
#define N 4

int main(){
    int matrice[M][N] = {
        {10, 20, 30, 40},
        {11, 21, 31, 41},
        {12, 22, 32, 42}
    };

    int i, j;
    for(i=0; i<M; i++){
        for(j=0; j<N; j++){
            printf("%d ", matrice[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

```
#include <stdio.h>
#define M 3
#define N 4

int main(){
    int matrice[M][N] = {
        {10, 20, 30, 40},
        {11, 21, 31, 41},
        {12, 22, 32, 42}
    };

    int i, j;
    for (i = 0; i < M; i++) {
        for (j = 0; j < N; j++) {
            printf("%d ", (*(matrice + i) + j));
        }
        printf("\n");
    }
    return 0;
}
```

Puntatori

```
#include <stdio.h>
#include <stdlib.h>

#define N 2

int main(){
    int m[N][N] = {{1,20},{300,4000}};

    printf("%p %p %p %p\n", m, m[0], m[0]+1, m+1);
    printf("%d %d %d %d\n", **m, *m[0], *(m[0]+1), **(m+1));

    return 0;
}
```

Cosa stampa?

indirizzo	memoria
?	1
?+4	20
?+8	300
?+12	4000

Puntatori

```
#include <stdio.h>
#include <stdlib.h>

#define N 2

int main(){
int m[N][N] = {{1,20},{300,4000}};

printf("%p %p %p %p\n",m,m[0],m[0]+1,m+1);
printf("%li %li %li %li\n",m,m[0],m[0]+1,m+1);
printf("%d %d %d %d\n",**m,*m[0],*(m[0]+1),**(m+1));

return 0;
}
```

indirizzo	memoria
?	1
?+4	20
?+8	300
?+12	4000

`m, m[0]` puntano allo stesso blocco di memoria, che contiene il primo elemento della matrice

```
[goldleaf@GoldBookPro2 L6 % ./a.out
0x16b8e35b0 0x16b8e35b0 0x16b8e35b4 0x16b8e35b8
6099449264 6099449264 6099449268 6099449272
1 1 20 300
```

4 perché ogni `int` in memoria è grande 4 byte

Puntatori

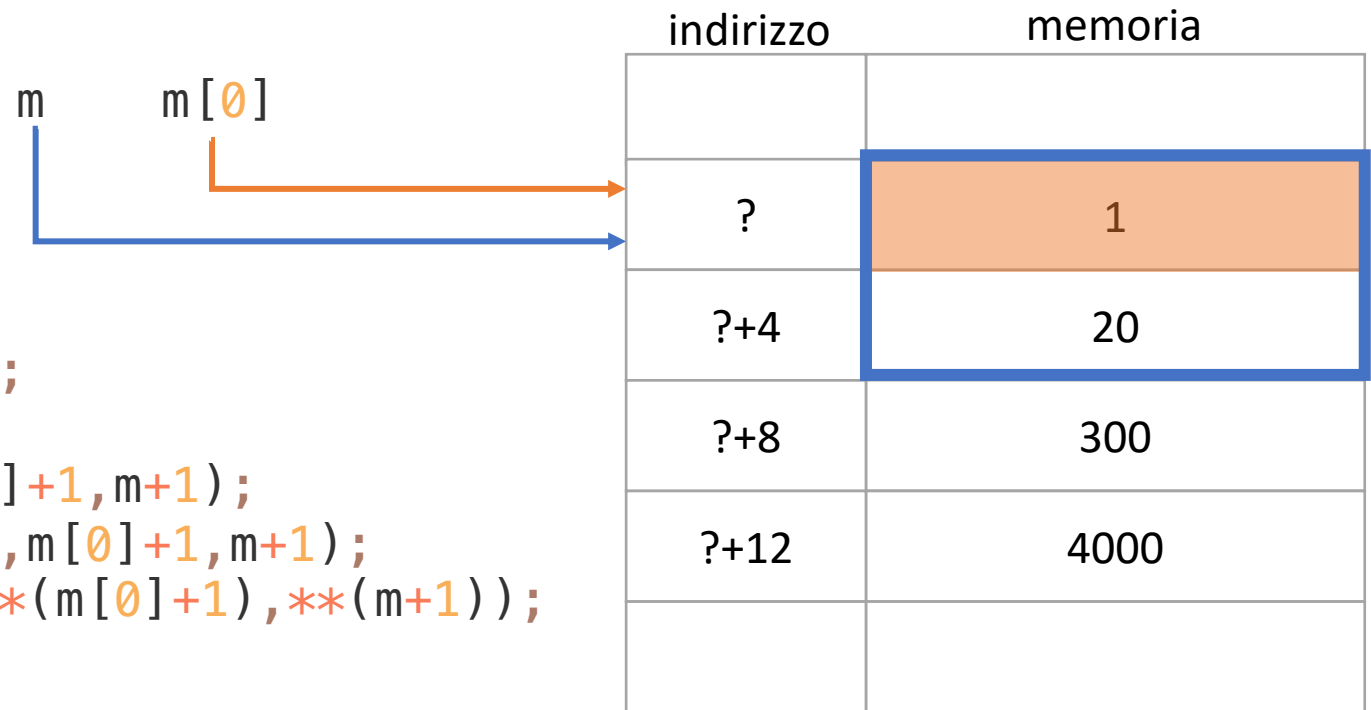
```
#include <stdio.h>
#include <stdlib.h>

#define N 2

int main(){
    int m[N][N] = {{1,20},{300,4000}};

    printf("%p %p %p %p\n",m,m[0],m[0]+1,m+1);
    printf("%li %li %li %li\n",m,m[0],m[0]+1,m+1);
    printf("%d %d %d %d\n",**m,*m[0],*(m[0]+1),**(m+1));

    return 0;
}
```



m punta ad un oggetto grande come una intera riga della matrice ($2 * \text{sizeof}(\text{int})$)
m[0] punta ad un oggetto grande come una cella della matrice (int)

```
[goldleaf@GoldBookPro2 L6 % ./a.out
0x16b8e35b0 0x16b8e35b0 0x16b8e35b4 0x16b8e35b8
6099449264 6099449264 6099449268 6099449272
1 1 20 300
```

Puntatori

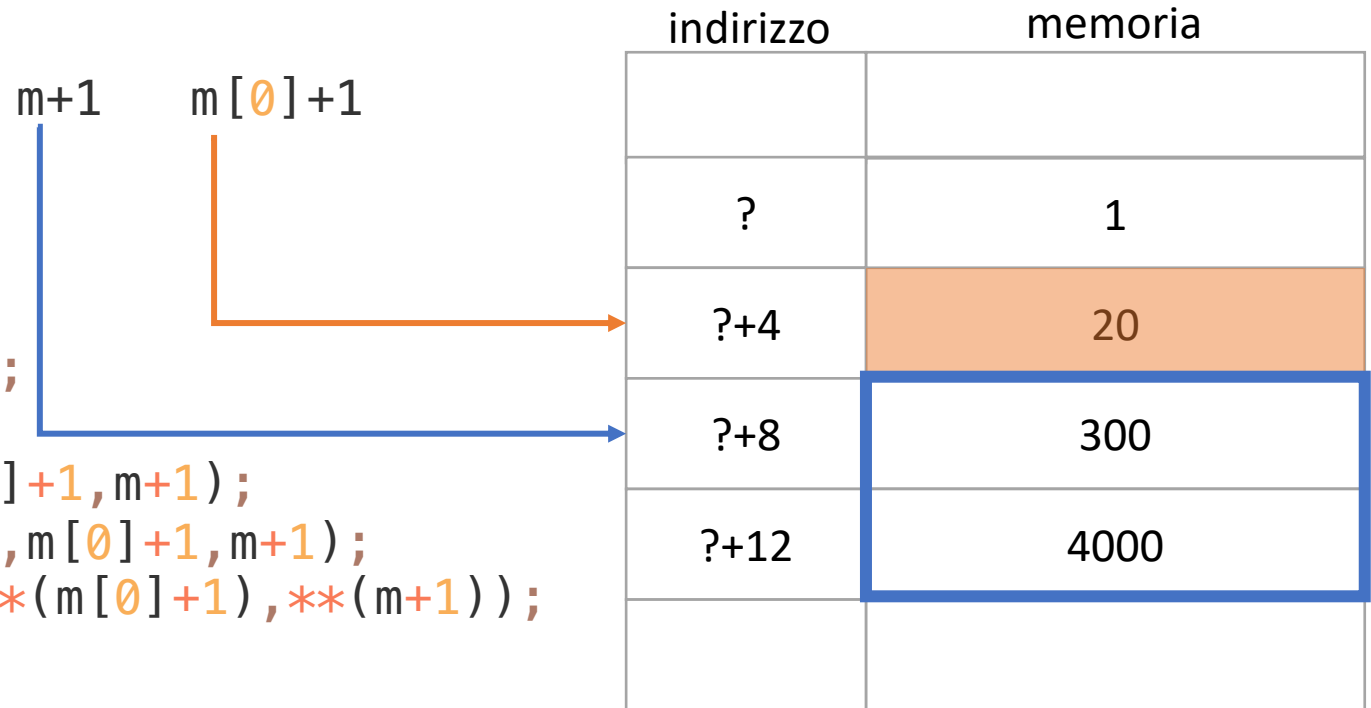
```
#include <stdio.h>
#include <stdlib.h>

#define N 2

int main(){
    int m[N][N] = {{1,20},{300,4000}};

    printf("%p %p %p %p\n",m,m[0],m[0]+1,m+1);
    printf("%li %li %li %li\n",m,m[0],m[0]+1,m+1);
    printf("%d %d %d %d\n",**m,*m[0],*(m[0]+1),**(m+1));

    return 0;
}
```



indirizzo	memoria
?	1
?+4	20
?+8	300
?+12	4000

m punta ad un oggetto grande come una intera riga della matrice ($2 * \text{sizeof}(\text{int})$)
m[0] punta ad un oggetto grande come una cella della matrice (int)

```
[goldleaf@GoldBookPro2 L6 % ./a.out
0x16b8e35b0 0x16b8e35b0 0x16b8e35b4 0x16b8e35b8
6099449264 6099449264 6099449268 6099449272
1 1 20 300
```

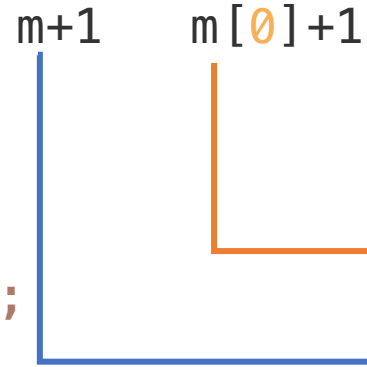

Puntatori

```
#include <stdio.h>
#include <stdlib.h>
```

```
#define N 2
```

```
int main(){
int m[N][N] = {{1,20},{300,4000}};

printf("%p %p %p %p\n",m,m[0],m[0]+1,m+1);
printf("%li %li %li %li\n",m,m[0],m[0]+1,m+1);
printf("%d %d %d %d\n",**m,**m[0],*(m[0]+1),**m+1);
```



indirizzo	memoria
?	1
?+4	20
?+8	300
?+12	4000

```
printf("%li, %li\n", (long)m[0]+1 - (long)m[0], (long)m+1 - (long)m );
```

```
int *p = &m[0][0];
for (int i=0; i<N*N; i++)
    printf("%p: %d\n",p+i,*(p+i));
return 0;
}
```

Puntatori

```
#include <stdio.h>
#include <stdlib.h>
```

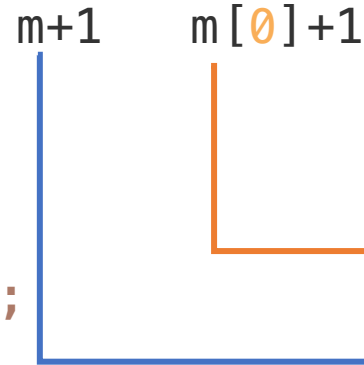
```
#define N 2
```

```
int main(){
int m[N][N] = {{1,20},{300,4000}};

printf("%p %p %p %p\n",m,m[0],m[0]+1,m+1);
printf("%li %li %li %li\n",m,m[0],m[0]+1,m+1);
printf("%d %d %d %d\n",**m,*m[0],*(m[0]+1),**(m+1));
```

```
printf("%li, %li\n", (long)(m[0]+1) - (long)m[0],
(long)(m+1) - (long)m );
```

```
int *p = &m[0][0];
for (int i=0; i<N*N; i++)
    printf("%p: %d\n",p+i,*(p+i));
return 0;
}
```



indirizzo	memoria
?	1
?+4	20
?+8	300
?+12	4000

```
[goldleaf@GoldBookPro2 L6 % ./a.out
0x16b8e35b0 0x16b8e35b0 0x16b8e35b4 0x16b8e35b8
6099449264 6099449264 6099449268 6099449272
1 1 20 300
4, 8
0x16b8e35b0: 1
0x16b8e35b4: 20
0x16b8e35b8: 300
0x16b8e35bc: 4000
```

Puntatori

```
#include <stdio.h>
#include <stdlib.h>

#define N 2

int main(){
    int m2[N*N] = {1,20,300,4000};

    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            printf("%p: %d\n", m2+(i*N+j), *(m2+(i*N+j)));

    return 0;
}
```

indirizzo	memoria
?	1
?+4	20
?+8	300
?+12	4000

In memoria hanno lo stesso layout, cambia l'accesso e come vengono gestiti i puntatori

Array di puntatori

ESERCIZIO 8

Ordinamento delle stringhe per lunghezza.

Tempo: 40 min.

Scrivete un programma che chieda all'utente di inserire 5 stringhe, e le memorizzi in un array di puntatori a `char` appropriatamente dimensionato. Il programma chiama poi una funzione di prototipo

```
void ordina(char **s, int lung)
```

il cui primo parametro rappresenta l'array di puntatori a `char`, e il cui secondo parametro rappresenta il numero di stringhe puntate dagli elementi dell'array (in questo esempio, 5). La funzione ordina l'array di stringhe in ordine non decrescente di lunghezza. Scrivete una funzione ausiliaria per il calcolo della lunghezza di una stringa. Come algoritmo di ordinamento usate Bubble Sort, che avete implementato nell'Esercizio 11 della Lezione 4.

str[0]	ciao
str[1]	abaco
str[2]	puntatore
str[3]	stringa
str[4]	ricorsione

```

int main() {
    char buffer[MAX_LENGTH];
    char strings[NUM_STRINGS][MAX_LENGTH];

    // Acquisizione delle stringhe dall'utente
    for (int i = 0; i < NUM_STRINGS; i++) {
        printf("Inserisci la stringa %d: ", i + 1);
        fgets(buffer, MAX_LENGTH, stdin);
        buffer[strcspn(buffer, "\n")] = '\0';

        strcpy(strings[i], buffer);
    }

    // Ordina le stringhe per lunghezza
    ordina(strings, NUM_STRINGS);

    // Stampa le stringhe ordinate
    printf("\nStringhe ordinate per lunghezza:\n");
    for (int i = 0; i < NUM_STRINGS; i++) {
        printf("%s\n", strings[i]);
    }

    return 0;
}

```

```
void ordina(char **s, int lung)
```

str[0]	ciao
str[1]	abaco
str[2]	puntatore
str[3]	stringa
str[4]	ricorsione

```

int main() {
    char strings[5][100] = {"abaco", "puntatore", "stringa", "ricorsione"};

    // Ordina le stringhe per lunghezza
    ordina(strings, NUM_STRINGS);

    // Stampa le stringhe ordinate
    printf("\nStringhe ordinate per lunghezza:\n");
    for (int i = 0; i < NUM_STRINGS; i++) {
        printf("%s\n", strings[i]);
    }

    return 0;
}

```

es9errore.c:50:12: warning: incompatible pointer types passing 'char[5][100]' to parameter of type 'char **' [-Wincompatible-pointer-types]

ordina(strings, NUM_STRINGS);

es9errore.c:22:20: note: passing argument to parameter 's' here

void ordina(char **s, int lung) {

1 warning generated.

fgets(buffer, MAX_LENGTH, stdin);

buffer[strcspn(buffer, "\n")] = '\0';

strcpy(strings[i], buffer);

str[1] abaco

str[2] puntatore

str[3] stringa

str[4] ricorsione

void ordina(char **s, int lung)

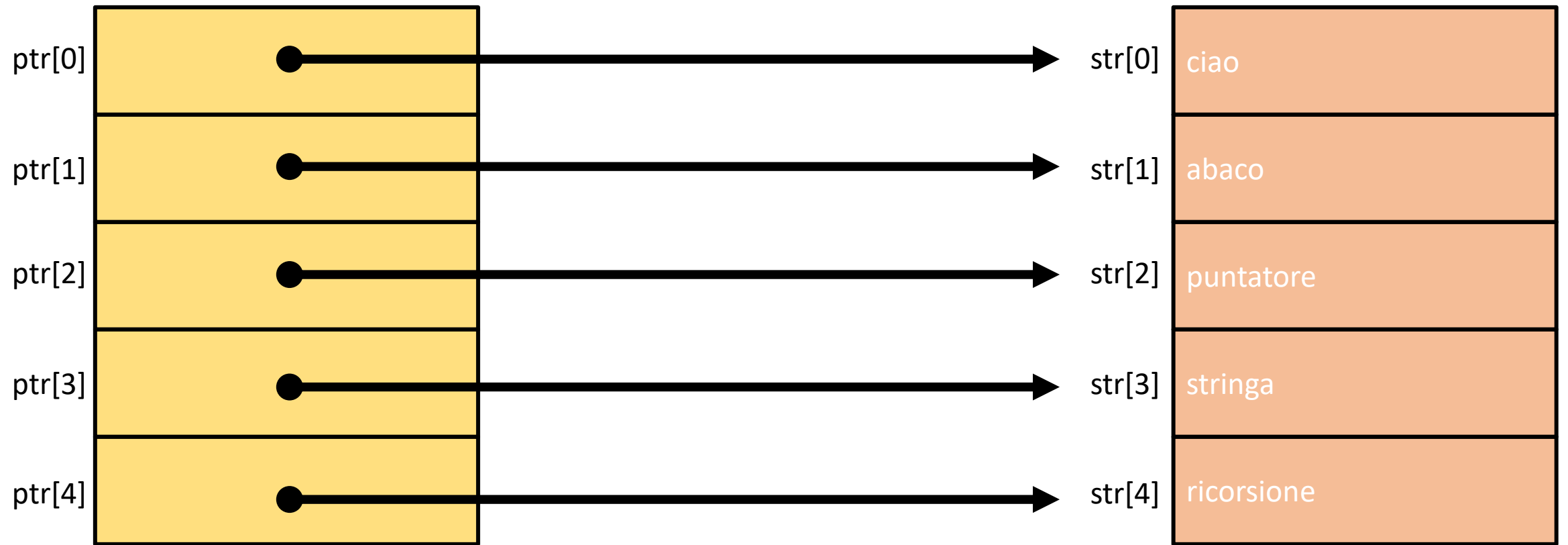
```
char *ptrs[NUM_STRINGS];
char buffer[MAX_LENGTH];
char strings[NUM_STRINGS][MAX_LENGTH];

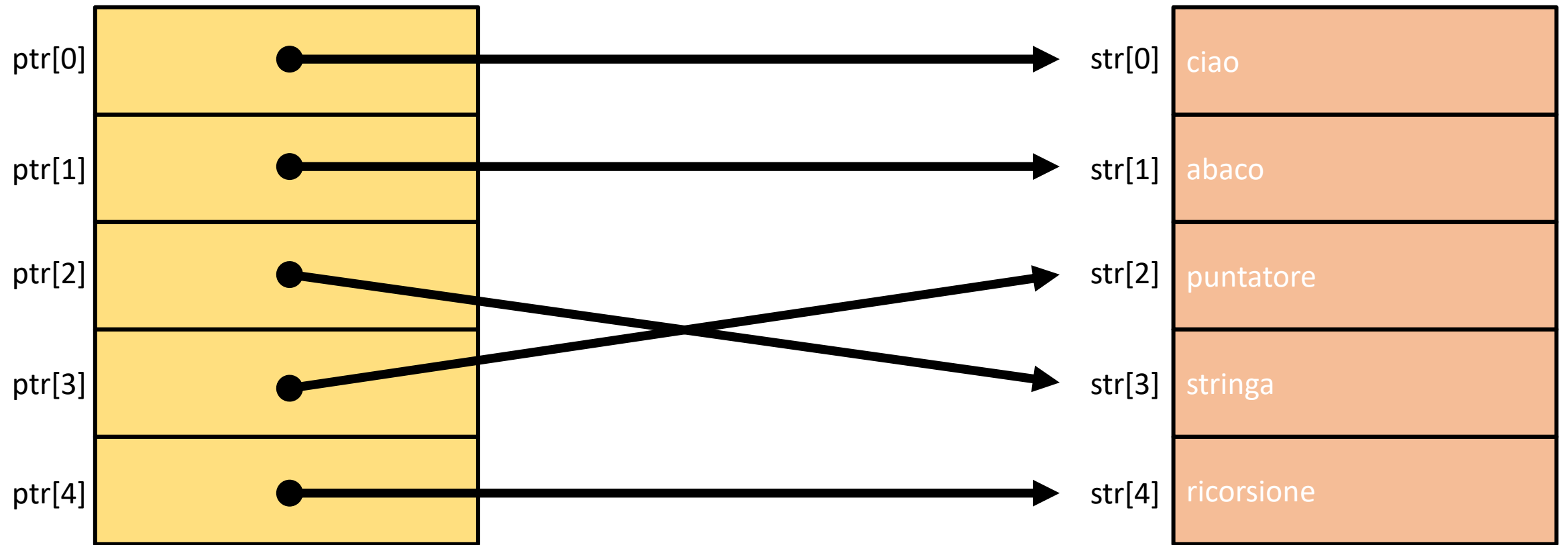
// Acquisizione delle stringhe dall'utente
for (int i = 0; i < NUM_STRINGS; i++) {
    printf("Inserisci la stringa %d: ", i + 1);
    fgets(buffer, MAX_LENGTH, stdin);
    buffer[strcspn(buffer, "\n")] = '\0';

    strcpy(strings[i], buffer);
    ptrs[i] = strings[i];
}

// Ordina le stringhe per lunghezza
ordina(ptrs, NUM_STRINGS);
```

str[0]	ciao
str[1]	abaco
str[2]	puntatore
str[3]	stringa
str[4]	ricorsione





Funzioni e Puntatori

- Le funzioni in C possono avere al più un output (return)
- E se dobbiamo scrivere una funzione che restituisce più valori?

Funzioni e Puntatori

- Le funzioni in C possono avere un output al più
- E se dobbiamo scrivere una funzione che restituisce più valori?
 - usiamo i puntatori nei parametri in ingresso, in modo che la funzione memorizzi qualcosa all'indirizzo indicato dai puntatori
 - Conoscete una funzione che lo fa?

Funzioni e Puntatori

- Le funzioni in C possono avere un output al più
- E se dobbiamo scrivere una funzione che restituisce più valori?
 - usiamo i puntatori nei parametri in ingresso, in modo che la funzione memorizzi qualcosa all'indirizzo indicato dai puntatori
 - Conoscete una funzione che lo fa? `scanf`
 - Prende in ingresso dei puntatori perché *scrive il valore letto all'indirizzo indicato dai puntatori*
 - Per l'equivalenza array – puntatori, non serve specificare `&` quando passiamo a `scanf` una stringa (cioè `char []`)

Struct

- Usate per memorizzare elementi diversi ma legati tra loro e riferiti allo stesso “concetto”
- Evitiamo di “portarci in giro” più variabili di tipo diverso, che usiamo insieme perché riferite allo stesso concetto

Struct

- Usate per memorizzare elementi diversi ma legati tra loro e riferiti allo stesso “concetto”
- Evitiamo di “portarci in giro” più variabili di tipo diverso, che usiamo insieme perché riferite allo stesso concetto

```
char first_name[N];  
char last_name[N];  
char ssn[N];
```

Struct

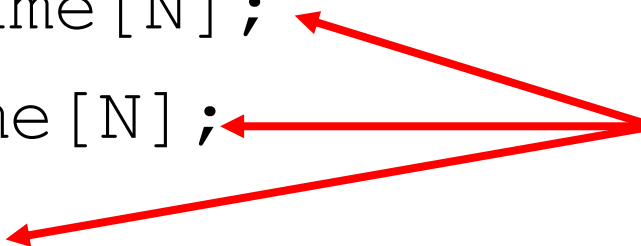
- Usate per memorizzare elementi diversi ma legati tra loro e riferiti allo stesso “concetto”
- Evitiamo di “portarci in giro” più variabili di tipo diverso, che usiamo insieme perché riferite allo stesso concetto

```
char first_name[N];
```

```
char last_name[N];
```

```
char ssn[N];
```

Variabili legate perché riferite allo stesso concetto (persona)



Struct

- Usate per memorizzare elementi diversi ma legati tra loro e riferiti allo stesso “concetto”
- Evitiamo di “portarci in giro” più variabili di tipo diverso, che usiamo insieme perché riferite allo stesso concetto

```
struct persona {  
    char first_name[N];  
    char last_name[N];  
    char ssn[N];  
};
```

```
struct persona persona1;  
struct persona persona2;
```

Variabili legate perché riferite allo stesso concetto (persona)

Definiamo un nuovo tipo `struct persona`, possiamo dichiarare variabili di questo tipo

Struct

- Usate per memorizzare elementi diversi ma legati tra loro e riferiti allo stesso “concetto”
- Evitiamo di “portarci in giro” più variabili di tipo diverso che usiamo insieme perché riferite allo stesso concetto

```
struct persona {  
    char first_name[N];  
    char last_name[N];  
    char ssn[N];  
}
```

Variabili legate perché riferite allo stesso concetto (persona)

Definiamo un nuovo tipo `struct persona`, possiamo dichiarare variabili di questo tipo

```
struct persona persona1;  
struct persona persona2;
```



2 dichiarazioni anziché 6

Dichiarazione Struct

Dichiarazione con “tag di struttura”

```
struct persona {  
    char first_name[N];  
    char last_name[N];  
    char ssn[N];  
};
```

Dichiarazione con con alias
(typedef)

```
typedef struct {  
    char first_name[N];  
    char last_name[N];  
    char ssn[N];  
} persona;
```

Dichiarazione Struct

Dichiarazione con “tag di struttura”

```
struct persona {  
    char first_name[N];  
    char last_name[N];  
    char ssn[N];  
};
```

```
struct persona personal;  
struct persona persona2;
```

Dichiarazione con alias (typedef)

```
typedef struct {  
    char first_name[N];  
    char last_name[N];  
    char ssn[N];  
} persona;
```

```
persona personal;  
persona persona2;
```

La dichiarazione viene fatta fuori dal main, in modo che sia visibile nell'intero codice

Struct e Padding

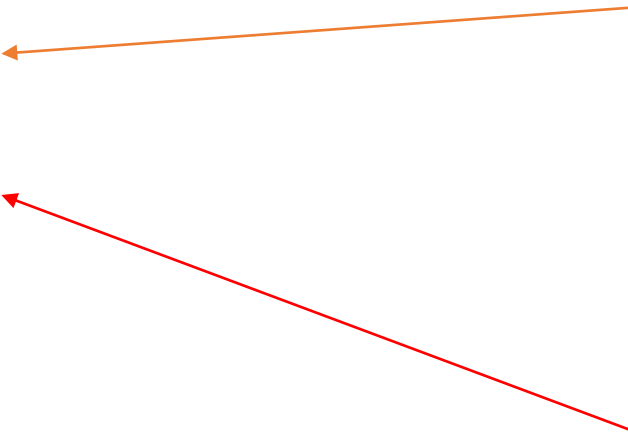
- In generale, ogni variabile viene memorizzata ad un indirizzo *divisibile per la sua dimensione*
- Variabili di tipo `int` (4 byte) sono memorizzate a un indirizzo divisibile per 4
- Variabili di tipo puntatore (8 byte) sono memorizzate a un indirizzo divisibile per 8
- Variabili di tipo `struct` seguono l'allineamento del campo (semplice) di dimensione maggiore
 - Il compilatore aggiunge alcune byte di *padding* per allineare la `struct` in memoria e rendere l'accesso più efficiente
- NOTA: placeholder `%zu` per il tipo restituito da `sizeof`

Struct e Padding

```
struct esempio {  
    char c; // 1 byte  
    int x;  // 4 byte  
    char d; // 1 byte  
};
```

Struct e Padding

```
struct esempio {  
    char c; // 1 byte  
    int x;  // 4 byte  
    char d; // 1 byte  
};
```



Aggiunta di 3 byte di padding, così
x viene memorizzata ad un
indirizzo divisibile per 4
→ x “finisce” al byte 8

Quindi d “finisce” al byte 9
→ Aggiunta di 3 byte di padding,
così la struct occupa 12 byte, cioè
una dimensione divisibile per la
dimensione del campo semplice
più grande (cioè x)

Struct e Padding

```
struct esempio {  
    char c; // 1 byte  
    int x;  // 4 byte  
    char d; // 1 byte  
};
```



```
struct esempio_padded {  
    char c;  
    char pad1[3];  
    int x;  
    char d;  
    char pad2[3];  
};
```

Aggiunta di 3 byte di padding, così
x viene memorizzata ad un
indirizzo divisibile per 4
→ x “finisce” al byte 8

Quindi d “finisce” al byte 9
→ Aggiunta di 3 byte di padding,
così la struct occupa 12 byte, cioè
una dimensione divisibile per la
dimensione del campo semplice
più grande (cioè x)

Parametri a Linea di Comando

- Il prototipo corretto della funzione `main` è:
 - `int main(int argc, char *argv[])`
- `argc` e `argv` modellano i parametri a linea di comando, ovvero un array di stringhe
 - `argc`: numero di parametri
 - `argv`: array di puntatori a stringhe, ogni stringa è un parametro

Parametri a Linea di Comando

```
gcc -o eseguibile sorgente.c
```

Parametri a Linea di Comando

```
gcc -o eseguibile sorgente.c
```

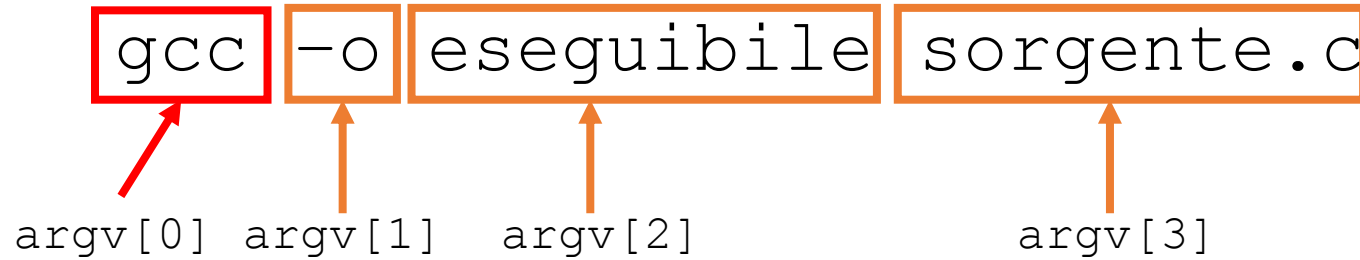


Nome del programma
che stiamo eseguendo



Parametri a linea di comando (input del programma che
specifichiamo al momento dell'esecuzione)
→ Una volta lanciato, l'esecuzione è non-interattiva
(comodo per scripting)

Parametri a Linea di Comando



```
gcc -o eseguibile sorgente.c
```

`argv[0]` `argv[1]` `argv[2]` `argv[3]`

Quindi `argc` vale 4

`argv[0]` contiene il percorso del programma che stiamo eseguendo

Parametri a Linea di Comando

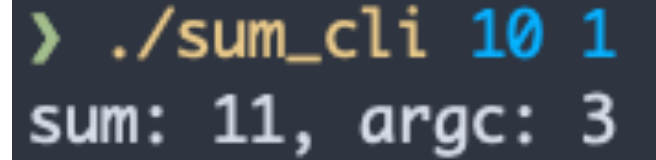
```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]){
    int n1, n2, result;
    result = atoi(argv[1]) + atoi(argv[2]);
    printf("sum: %d, argc: %d\n", result, argc);
    return 0;
}
```

Parametri a Linea di Comando

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]){
    int n1, n2, result;
    result = atoi(argv[1]) + atoi(argv[2]);
    printf("sum: %d, argc: %d\n", result, argc);
    return 0;
}
```

A terminal window with a dark background. The first line shows a prompt character followed by the command './sum_cli 10 1'. The second line shows the output 'sum: 11, argc: 3'.

```
> ./sum_cli 10 1
sum: 11, argc: 3
```

Parametri a Linea di Comando

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main(int argc, char *argv[]){
    int n1, n2, result;
    result = atoi(argv[1]) + atoi(argv[2]);
    printf("sum: %d, argc: %d\n", result, argc);
    return 0;
}
```



```
> ./sum_cli 10 1
sum: 11, argc: 3
```

Non so se l'utente che
esegue il mio programma
specificherà i parametri che
mi aspetto di ricevere

Devo verificare!

```
> ./sum_cli
fish: Job 1, './sum_cli' terminated by signal SIGSEGV (Address boundary error)
```

Parametri a Linea di Comando

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]){
    int n1, n2, result;
    if (argc != 3){
        printf("Mi servono due param!\n");
        return -1;
    }
    result = atoi(argv[1]) + atoi(argv[2]);
    printf("sum: %d, argc: %d\n", result, argc);
    return 0;
}
```



Non so se l'utente che
esegue il mio programma
specificherà i parametri che
mi aspetto di ricevere

```
> ./sum_cli_robust
Mi servono due param!
```

Redirezione `stdin/stdout/stderr`

- “Redirect”: funzionalità implementata dal terminale e dal sistema operativo per modificare `stdin/stdout/stderr` e farli puntare a dei file senza modificare il programma
- Redirezione `stdin`: l’input da tastiera letto dal programma viene sostituito dalla lettura di un file
- Redirezione `stdout`: l’output scritto su terminale viene sostituito dalla scrittura su un file
- Redirezione `stderr`: l’output scritto su terminale in presenza di errori viene sostituito dalla scrittura su un file

Redirezione `stdin/stdout/stderr`

- E' implementata a livello di terminale e sistema operativo
 - Quindi trasparente al programma

- Sintassi:

```
./myprogram [<argv>] <file-input >file-output 2> file-output
```

Redirezione `stdin/stdout/stderr`

- E' implementata a livello di terminale e sistema operativo
 - Quindi trasparente al programma
- Sintassi:

```
./myprogram argv <file-input >file-output 2>file-output
```

Invocazione standard del
programma con eventuale
`argv`

Nome del file che
sostituisce input da
tastiera

Nome del file che
sostituisce output su
terminale

Nome del file che
sostituisce output
errori su terminale

Redirezione stdin

- “Redirect”: funzionalità implementata dal terminale e dal sistema operativo per modificare `stdin/stdout/stderr` e farli puntare a dei file senza modificare il programma
- Redirezione `stdin`: l’input da tastiera letto dal programma viene sostituito dalla lettura di un file

```
#include <stdio.h>

int main(){
    int n1, n2;
    printf("Inserisci due numeri: ");
    scanf("%d %d", &n1, &n2);
    printf("Il risultato e': %d\n", n1 + n2);
    return 0;
}
```

```
> ./redirect
Inserisci due numeri: 11 22
Il risultato e': 33
```

Redirezione stdin

- “Redirect”: funzionalità implementata dal terminale e dal sistema operativo per modificare `stdin/stdout/stderr` e farli puntare a dei file senza modificare il programma
- Redirezione `stdin`: l’input da tastiera letto dal programma viene sostituito dalla lettura di un file

```
#include <stdio.h>

int main(){
    int n1, n2;
    printf("Inserisci due numeri: ");
    scanf("%d %d", &n1, &n2);
    printf("Il risultato e': %d\n", n1 + n2);
    return 0;
}
```

redirect_stdin.txt

```
1    11 22
2
```

```
> ./redirect <redirect_stdin.txt
Inserisci due numeri: Il risultato e': 33
```

Redirezione stdout

- “Redirect”: funzionalità implementata dal terminale e dal sistema operativo per modificare `stdin/stdout/stderr` e farli puntare a dei file senza modificare il programma
- Redirezione `stdout`: l’output scritto su terminale viene sostituito dalla scrittura su un file

```
#include <stdio.h>
```

```
int main(){  
    int n1, n2;  
    printf("Inserisci due numeri: ");  
    scanf("%d %d", &n1, &n2);  
    printf("Il risultato e': %d\n", n1 + n2);  
    return 0;  
}
```

```
> ./redirect >redirect_stdout.txt  
11 22
```

redirect_stdout.txt

```
1   Inserisci due numeri: Il risultato e': 33  
2   |
```

Redirezione stderr

- `stderr` è uno stream che viene usato per scrivere eventuali errori
 - di default, `stdout` e `stderr` scrivono nello stesso posto (il terminale) ma sono due stream separati

```
#include <stdio.h>
```

```
int main(){  
    int n1, n2;  
    printf("Inserisci due numeri: ");  
    scanf("%d %d", &n1, &n2);  
    if (n1 == 0){  
        fprintf(stderr, "n1 dev'essere != 0!\n");  
        return -1;  
    }  
    printf("Il risultato e': %d\n", n1 + n2);  
    return 0;  
}
```

```
> ./redirect_v2  
Inserisci due numeri: 11 22  
Il risultato e': 33
```

Redirezione stderr

- `stderr` è uno stream che viene usato per scrivere eventuali errori
 - di default, `stdout` e `stderr` scrivono nello stesso posto (il terminale) ma sono due stream separati

```
#include <stdio.h>
```

```
int main(){  
    int n1, n2;  
    printf("Inserisci due numeri: ");  
    scanf("%d %d", &n1, &n2);  
    if (n1 == 0){  
        fprintf(stderr, "n1 dev'essere != 0!\n");  
        return -1;  
    }  
    printf("Il risultato e': %d\n", n1 + n2);  
    return 0;  
}
```

```
> ./redirect_v2  
Inserisci due numeri: 0 22  
n1 dev'essere != 0!
```

Redirezione stderr

- `stderr` è uno stream che viene usato per scrivere eventuali errori
 - di default, `stdout` e `stderr` scrivono nello stesso posto (il terminale) ma sono due stream separati

```
#include <stdio.h>
```

```
int main(){  
    int n1, n2;  
    printf("Inserisci due numeri: ");  
    scanf("%d %d", &n1, &n2);  
    if (n1 == 0){  
        fprintf(stderr, "n1 dev'essere != 0!\n");  
        return -1;  
    }  
    printf("Il risultato e': %d\n", n1 + n2);  
    return 0;  
}
```

```
> ./redirect_v2 2>redirect_stderr.txt  
Inserisci due numeri: 11 22  
Il risultato e': 33
```


Redirezione stderr

- `stderr` è uno stream che viene usato per scrivere eventuali errori
 - di default, `stdout` e `stderr` scrivono nello stesso posto (il terminale) ma sono due stream separati

```
#include <stdio.h>
```

```
int main(){  
    int n1, n2;  
    printf("Inserisci due numeri: ");  
    scanf("%d %d", &n1, &n2);  
    if (n1 == 0){  
        fprintf(stderr, "n1 dev'essere != 0!\n");  
        return -1;  
    }  
    printf("Il risultato e': %d\n", n1 + n2);  
    return 0;  
}
```

```
> ./redirect_v2 2>redirect_stderr.txt  
Inserisci due numeri: 0 22
```

redirect_stderr.txt

```
1 |n1 dev'essere != 0!  
2
```

Redirezione stdout

- Il carattere > sovrascrive il contenuto nel file destinazione

```
> ./redirect >redirect_stdout.txt  
10 20  
> ./redirect >redirect_stdout.txt  
11 22
```

redirect_stdout.txt

```
1   Inserisci due numeri: Il risultato e': 33  
2   |
```

- Il carattere >> appende in coda al file destinazione

```
> ./redirect >>redirect_stdout.txt  
10 20  
> ./redirect >>redirect_stdout.txt  
11 22
```

redirect_stdout.txt

```
1   Inserisci due numeri: Il risultato e': 30  
2   Inserisci due numeri: Il risultato e': 33  
3   |
```

Redirezione stderr

- Il carattere `2>` sovrascrive il contenuto nel file destinazione

```
> ./redirect_v2 2>redirect_stderr.txt
Inserisci due numeri: 0 22
> ./redirect_v2 2>redirect_stderr.txt
Inserisci due numeri: 0 11
```

redirect_stderr.txt

1 n1 dev'essere != 0!

2

- Il carattere `2>>` appende in coda al file destinazione

```
> ./redirect_v2 2>>redirect_stderr.txt
Inserisci due numeri: 0 22
> ./redirect_v2 2>>redirect_stderr.txt
Inserisci due numeri: 0 11
```

redirect_sterr.txt

1 n1 dev'essere != 0!

2 n1 dev'essere != 0!

3

Esempio: popolare una matrice usando la redirectione
Step 1, creo un file

```
#define N 25
int main()
{

    for(int i=0; i < N; i++)    ./a.out > in.txt
        printf("%d \n", i+1);
    return 0;
}
```

Faccio un semplice `main`, e lo uso per creare un file `in.txt` che contiene 25 valori

Esempio: popolare una matrice usando la redirectione

Step 2, uso il file

```
#define N 5
int main()
{
    int matrix[N][N];

    for(int i=0; i<N; i++){
        for(int j=0; j<N; j++){
            scanf("%d",&matrix[i][j]);
        }
    }

    for(int i=0; i<N; i++){
        for(int j=0; j<N; j++){
            printf("%d ",matrix[i][j]);
        }
        printf("\n");
    }
}
```

Uso il file creato allo step precedente per popolare la matrice senza inserire a mano 25 valori.

Se voglio, posso editare il file `in.txt`

```
./a.out < in.txt
```

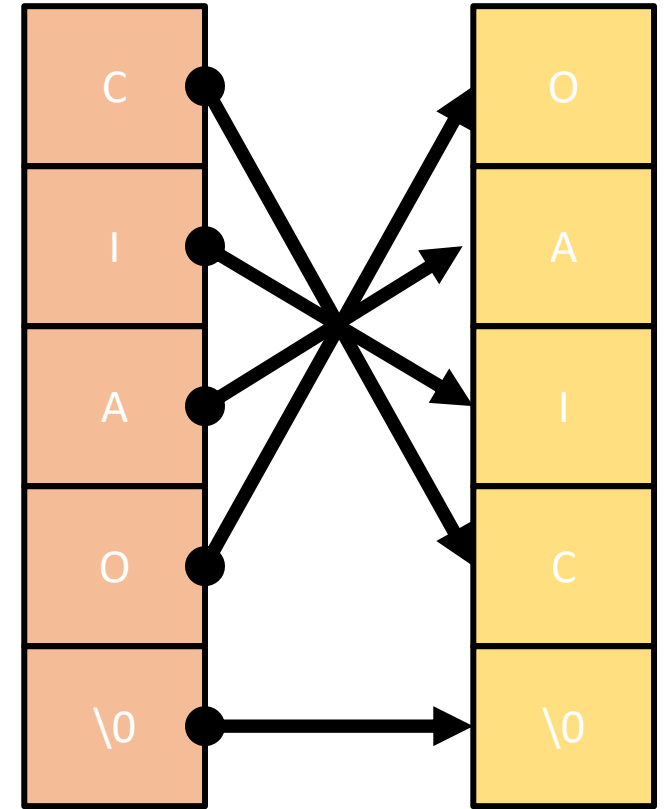
```
goldleaf@GoldBookPro2 L8 % ./a.out < in.txt
1 2 3 4 5
6 7 8 9 10
11 12 13 14 15
16 17 18 19 20
21 22 23 24 25
```

Trova l'Errore

Scrivete un programma che chieda all'utente di inserire una stringa, invochi una funzione che restituisce una copia della stringa scritta in ordine inverso, la visualizzi e termini. La funzione che inverte la stringa ha prototipo

```
void reverse(char *s, char *t)
```

dove il primo parametro rappresenta la stringa da invertire, e il secondo la stringa invertita. In `main`, invocate la funzione passandole come primo parametro (il nome di un) array di tipo `char` inizializzato con una costante stringa — ossia la stringa letta dall'utente — e come secondo parametro (il nome di un) array di tipo `char` della medesima dimensione del primo. Un esempio di come potrebbe essere scritta la funzione `main` del programma è in Fig. 1.

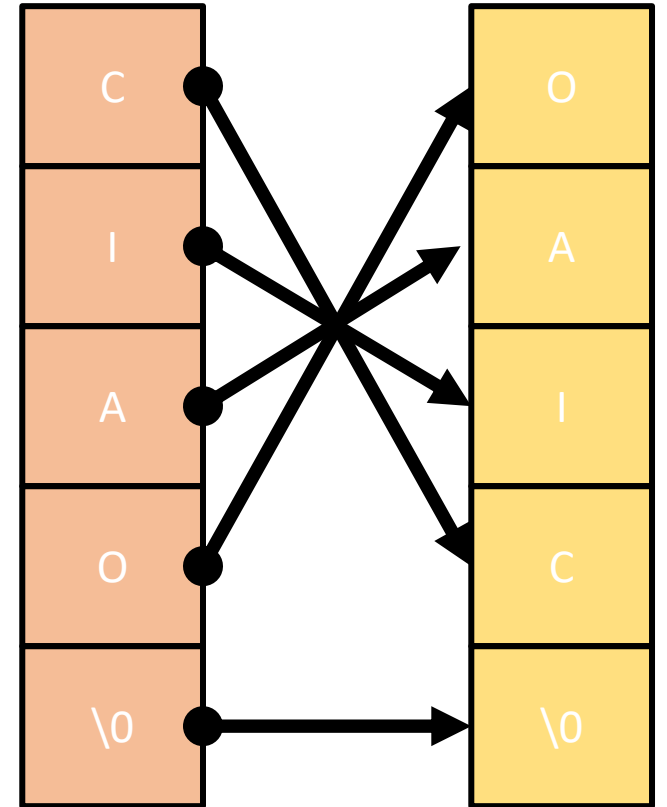


Trova l'Errore

```
void reverseEasy(char *s, char* srev)
{
    int length, c;
    char *end = s;
    length = strlen(s);

    for (c = 0; c < length - 1; c++)
        end++;

    for (c = 0; c < length; c++)
    {
        *srev = *end;
        srev++;
        end--;
    }
}
```



Trova l'Errore – 2

```
void reverse (char * s, char * t){  
  
    int i = 0, j = 0;  
    int cont = 0;  
    while(*(s+i) != '\0')  
        i++;  
  
    while((t[j] = s[i - 1])){  
        j++;  
        i--;  
        cont ++;  
    }  
}
```

